31ª Semana de Informática - UFV

Minicurso de Python

Apresentado por
Luis Felipe Martins Pereira



Ementa

- Introdução
- Conceitos de Linguagem de Programação
- História e Filosofia do Python (Zen of Python)
- Características da Linguagem
- Trade-Offs
- Aplicação no Mercado
- Atividade Prática
- Conclusão

Como Avaliar uma Linguagem de Programação?

- Legibilidade
- Redigibilidade
- Confiabilidade
- Ortogonalidade
- Suporte a abstrações
- Desempenho
- Curva de Aprendizado
- Ecossistema

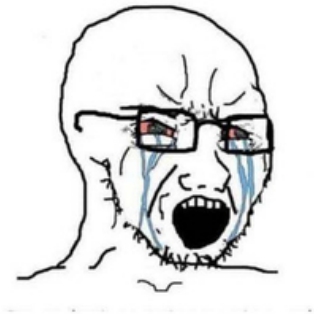
Legibilidade/Redigibilidade

Código em C

```c
for (int i = 0; i < 5; i++) {
    printf("%d\n", numeros[i]);
}
```

Código em Python

```python
for numero in numeros:
    print(numero)
```



Qual é mais legível?



Confiabilidade

Código em C

```c
int* p = new int(42);
int* q = p;

delete p;

cout << *q;
```

access after free!

Código em Python

```python
p = [42]
q = p

del p

print(q)
```

printa 42

História do Python



1989

Criador do Python, Guido Von Rossum, começa a desenvolver a linguagem

1990

A 1ª Edição pública da linguagem é lançada, de maneira open-source. Nasce Python 1.0

2000

Criação do **Zen of Python** um ano antes. Python 2.0 chega com Mudanças que consolidaram ainda mais a identidade da linguagem e ecossistema

2008

Grande reformulação da linguagem, Python 3.0 é incompatível com sua versão anterior e as duas coexistem

Zen of Python

O que é? Conjunto de Filosofias da Linguagem, os propósitos que seu criador definiu em 1999. É um documento de 20 itens, como pode ser visto abaixo:

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Imagem tirada de: https://peps.python.org/pep-0020/

Exemplos de ‘Zens’ do Python muito importantes

01 Readability Counts: Partindo de uma filosofia “Code is read much more often than it is written”

02 Simple is better than complex: Por vezes Python irá preferir algo simples que abstrato

03 Explicit is better than implicit: O código deve deixar clara a intenção do programador

04 There should be one - and only one - obvious way to do something: Programadores devem chegar a códigos próximos ao escrever algo em Python

Dadas a História e as Filosofias: O que é Python?

- Linguagem de alto nível e de propósito geral: Alta abstração e aplicável em diversas áreas
- Multiparadigma: POO, Funcional, Procedural
- Dinamicamente Tipada: Não é necessário declarar o tipo, a linguagem é capaz de interpretar
- Interpretada/Mista Multiplataforma: O código Python é executado por um interpretador e pode rodar em diferentes SOs sem precisar alterar o código-fonte.
- Image/Diagram: Case studies or examples of successful career development programs.

Trade-Offs

Pontos Fortes

- Legibilidade
- Redigibilidade
- Portabilidade
- Confiabilidade
- Ortogonalidade
- Produtividade
- Ecossistema
- Curva de Aprendizado

Pontos Fracos

- Tipagem Dinâmica
- Consumo de Memória; Desempenho
- GIL e limitações de paralelismo em CPU



Por que aprender Python?

Python é uma linguagem requisitada em diversas áreas da computação e fora da computação, com curva de aprendizado pouco acentuada

Áreas em que Python é Muito Utilizado

- IA/Machine Learning & Data Science
- Desenvolvimento WEB & Aplicações do BackEnd, como servidores
- Prototipagem de Jogos com biblioteca PyGame
- Cybersegurança, Automação de testes de segurança e dev de ferramentas

Hora da Prática

Desafio Pokemon

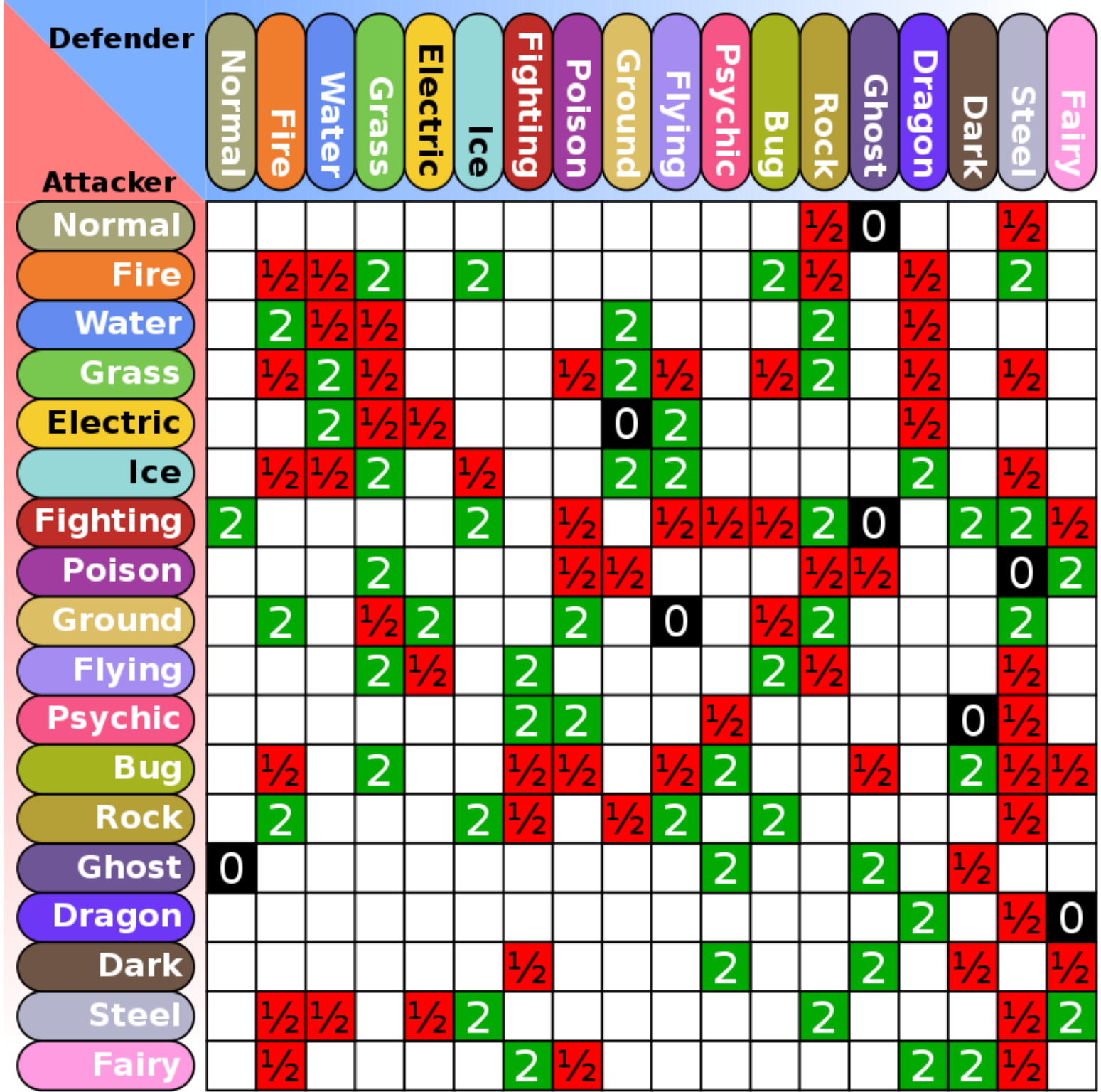
01 Implementar um Dicionário contendo os tipos do pokemon e seus multiplicadores de fraqueza

02 Implementar a classe pokemon conforme será especificado na documentação entregue

03 Implementar uma classe wrapper Combatente, que servirá para o pokemon em batala

04 Implementar a classe de Batalha para que você possa utilizar a interface de combate!

Dicionário de Fraquezas



Attacker \ Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													½	0			½	
Fire		½	½	2		2						2	½		½		2	
Water		2	½	½					2				2		½			
Grass		½	2	½				½	2	½		½	2		½		½	
Electric			2	½	½				0	2					½			
Ice		½	½	2		½			2	2					2		½	
Fighting	2					2		½		½	½	½	2	0		2	2	½
Poison				2				½	½				½	½			0	2
Ground		2		½	2			2		0		½	2				2	
Flying				2	½		2					2	½				½	
Psychic							2	2			½					0	½	
Bug		½		2			½	½		½	2			½		2	½	½
Rock		2				2	½		½	2		2					½	
Ghost	0										2			2		½		
Dragon															2		½	0
Dark							½				2			2		½		½
Steel		½	½		½	2							2				½	2
Fairy		½					2	½							2	2	½	

MUITO OBRIGADO!

MiniCurso Concluído

31ª Semana da Informática - UFV